THE CONVENTION-BASED APPROACH

Conventions are rules that must be applied to the design of certain components that are intended to work with a framework, so they behave in the expected manner. It might be that a class must be named in a particular way, so the framework can recognize it for what it is intended to be. This is the case, for example, with controller classes in MVC, which must have names that include `Controller` as the suffix. Or a convention might apply that specifies that a class designed for a specific use case must include a method that is named in a certain way and bears a predefined signature.

The two conventions that must be applied to a convention-based middleware class are (1) declaring a constructor that takes a `RequestDelegate` as a parameter representing the next middleware in the pipeline and (2) a method named `Invoke` or `InvokeAsync` that returns a `Task` and has at least one parameter—the first being an `HttpContext`.

To try this out, add a new class named `IpAddressMiddleware` to the application. The following example is added directly to the root of the project for simplicity. Replace the code with the next listing, which illustrates a middleware class that implements these conventions and logs the value of the visitor's IP address.

Listing 2.14 Middleware class based on the convention-based approach

```
namespace WebApplication1
{
    public class IpAddressMiddleware
    {
        private readonly RequestDelegate _next;
        public IpAddressMiddleware(RequestDelegate next) => _next =
next;                                                                  ❶

        public async Task InvokeAsync(HttpContext context,
        ➥ ILogger<IpAddressMiddleware> logger)                        ❷
```